

Machine Learning

16 notes across 4 week(s)

Week 1

Foundations of Machine Learning – Machine Learning (Module 1)

Learning Objectives

By the end of this video you will:

1. **Define** Machine Learning and its relationship to Data Science and Artificial Intelligence.
 2. **Identify** the key stages of a standard Machine Learning workflow (KDD, CRISP-DM).
 3. **Recognize** why ML is needed: data abundance vs. knowledge scarcity.
 4. **Describe** the evolution from empirical → theoretical → computational → data science age.
 5. **List** data types used in ML: structured, unstructured/semi-structured, time series, stream data.
-

Why Do We Need Machine Learning?

Data Everywhere, Knowledge Scarce

- **Physical → Digital:** Many real-world processes are now digital (photos, shopping, search, trading). This generates huge volumes of data.
- **Storage has grown:** From small disks (e.g. 6–8 GB in 2000) to phones with 128 GB and systems with petabytes. Data can be stored and processed.
- **Paradox:** We are **drowning in data** but **starving for knowledge**. Asking relevant questions (e.g. "Which stock to buy?", "How many orders are fake?") often does not yield good answers from raw data.
- **Role of ML:** Extract patterns and knowledge from massive data automatically, so we can answer such questions.

Evolution of Data and Science (Historical View)

Era	Period	Focus
Empirical science	Before ~1600	Ideas from observation (e.g. sun rises in east, sets in west).
Theoretical science	~1600– 1950s	Mathematical models and generalizations from observations.
Computational science	~1950s– 1990s	Limited computers; simulations and more complex models.
Data science / ML	Today	High compute + storage + connectivity; algorithms learn patterns from data instead of only hand-written rules.

What Is Machine Learning?

Definition: Machine Learning is a field of study that gives computers the ability to **learn from data without being explicitly programmed**.

- We do **not** hand-code all rules. The system **discovers** patterns and rules from data.
- ML has roots in **data mining**: extracting useful, previously unknown knowledge from large datasets.
- In ML we discover meaningful patterns **automatically or semi-automatically**.

The Machine Learning Pipeline (High-Level)

1. **Data + Questions** — Start with data and the question we want to answer (e.g. "Will it rain tomorrow?").
2. **Data selection** — Choose a subset of data relevant to the question (full data can be costly, slow, noisy).
3. **Data pre-processing** — Clean, transform, remove noise, handle missing values, etc.
4. **ML / data mining** — Run algorithms on the prepared data.
5. **Interpretation & evaluation** — Use performance metrics and tools to interpret results.
6. **Knowledge** — If we can answer the original question from this knowledge, the pipeline is successful; otherwise we revisit earlier stages.

Exam tip: Pipeline success = ability to answer the question you started with. If not, fix one or more stages (e.g. algorithm, data selection, cleaning).

Origins of Machine Learning

ML is **multidisciplinary**. It uses ideas from:

- **Statistics & AI** — Theory and prediction.
- **Pattern recognition** — Detecting structure in data.
- **Database systems** — Storing and accessing large data.

Machine Learning Workflows

ML is **not** “run one algorithm.” It is a **multi-step process**. Two standard frameworks:

1. KDD (Knowledge Discovery in Databases)

Stage	Activity
1	Raw data — Gather and store data (e.g. weather: date, temperature, humidity, wind).
2	Data integration & cleaning — Merge sources, clean, preprocess; build a data warehouse .
3	Data selection — From the warehouse, select task-relevant data .
4	Data mining / ML — Apply algorithms to find patterns.
5	Pattern evaluation — Measure quality of extracted patterns.
6	Knowledge — Final, actionable insights; check if original questions are answered.

2. CRISP-DM (Industry Framework)

Stage	Activity
1	Business understanding — Data, prior knowledge, questions.
2	Data preparation — Integrate and prepare data (integration, cleaning).
3	Modeling — Select and apply ML algorithms to build models.
4	Evaluation — Apply model on new data and evaluate performance.
5	Deployment / knowledge — Use model and insights; verify that questions are answered.

Business Intelligence View (Alternative 5-Step View)

1. Data sources → 2. Pre-processing (integration, warehousing) → 3. Data exploration → 4. ML/data mining → 5. Presentation & decisions.

Data Types in Machine Learning

Type	Description	Examples
Structured	Arranged and stored in a fixed structure	Tabular data, graphs (as structured representations).
Unstructured / semi-structured	No strict schema	News articles, web pages. Often converted to structured form before ML.
Time series	Values over time	Sensor logs, stock prices.
Advanced	Special handling	Stream data (real-time, e.g. IoT), socio-temporal , spatial data.

Summary

- **Why ML:** Lots of data, good storage and compute, but we need **knowledge**; ML extracts patterns from data.
- **What is ML:** Learning from data **without explicit programming**; evolved from data mining.
- **Pipeline:** Data + question → selection → pre-processing → ML → evaluation → knowledge.
- **Workflows:** KDD and CRISP-DM describe full lifecycle from raw data to knowledge.
- **Data types:** Structured, unstructured/semi-structured, time series, and stream/advanced types.

Use this note for definitions, pipeline stages, and exam-style questions on “why ML” and “what is ML.”

Supervised Learning – Learning from Labelled Data – Machine Learning (Module 1)

Learning Objectives

By the end of this video you will:

1. **Define** supervised learning and its core concepts (data, labels, model).
 2. **Distinguish** between **classification** and **regression**.
 3. **Describe** the process of **training** and **testing** a classification model.
 4. **Give** real-world examples of classification and regression.
-

Two Fundamental Types of Learning

Type	Also called	Goal	Data
Supervised learning	Prediction methods	Predict unknown values	Data with correct answers (labels)
Unsupervised learning	Descriptive methods	Find patterns and structure	Data without labels

What Is Supervised Learning?

- **Idea:** Learn from examples where the **correct answer (label)** is known, then **predict** for new inputs.
- **Labels** = correct answers = class labels = target. Terms are used interchangeably.
- **Training examples:** Pairs (**input features, desired output**). Input = **X**, output = **Y** (label/target).
- The model learns a **function f** that maps **X** → **Y**. At deployment we get only **X** and use **f** to predict **Y**.

Analogy: Student learns from a textbook with answers at the back (data + labels), then solves new questions (prediction).

Classification vs Regression

Both are **supervised**: we have **X** and **Y** and want to predict **Y** for new **X**.

	Classification	Regression
Output Y	Categorical (finite, discrete classes)	Continuous / numerical
Examples	Rain yes/no, spam/not spam, dog/cat, cheat yes/no	Petrol price, stock price, salary, vehicle mileage
Number of possible values	Fixed (e.g. 2, 3, 5)	Effectively infinite (any number in a range)

- **Classification:** Predict one of a **fixed set of categories** (e.g. spam vs not spam, malignant vs benign).
- **Regression:** Predict a **numeric value** (e.g. price, salary, distance).

Classification in More Detail

- Data is often in **tabular form**: rows = records (tuples), columns = **attributes**. One special attribute is the **class label** (target **Y**); the rest are **X**.
- **Goal:** Build a model that, for a **new or unseen** record (given only **X**), assigns a **class label** as accurately as possible.

Example – Cheat detection:

- Attributes: e.g. Refund (yes/no), Marital status (single/married/divorced), Taxable income.
- Class label: Cheat (yes/no).
- Training: Learn from many (X, Y) pairs. At test time: given only X (e.g. refund=no, marital=married, income=125K), predict cheat = yes or no.

Training and Testing a Classification Model

1. **Split data:** Divide the full dataset into two **disjoint** parts:
 - **Training set** — used to **build** the model **f**.
 - **Test set** — used to **evaluate** the model (never used for training).
2. **Training phase:**
 - Use **training data** (X and Y) and a **learning algorithm** to obtain the model **f**.
 - **f** captures the relationship between X and Y.
3. **Testing phase:**
 - Take **test data** (we have both X and true Y, but **ignore Y** for the moment).

- Feed **only X** from each test record into **f**; get **predicted Y**.
- **Compare** predicted Y with the **true Y** (which we had hidden).
- If they match often → model is good; if not → model needs improvement.

4. **Deployment:** Once performance is satisfactory, use **f** in the real world: input **X**, output predicted **Y**.

Exam tip: Training data → build **f**. Test data → measure performance; test set must be **independent** of training (no overlap). Never evaluate only on training data for final performance.

Real-World Applications (Recap)

Classification (discrete labels):

- Weather: rain vs no rain.
- Spam: spam vs not spam.
- Cheat detection: cheated vs not.
- Image: dog vs cat (or multi-class).
- Medical: malignant vs benign; disease vs no disease.

Regression (continuous output):

- Petrol/oil/gold/stock price.
- Salary given age/experience.
- Vehicle mileage (e.g. km/l).
- Stopping distance given speed.

Summary

- **Supervised learning** = learning from **(X, Y)** to predict **Y** for new **X**.
- **Classification:** Y is categorical (finite classes).
- **Regression:** Y is continuous/numerical.
- **Process:** Split data → train on training set → test on separate test set → deploy if good.
- Use this note for exam questions on definitions, classification vs regression, and train/test procedure.

Unsupervised Learning – Discovering Hidden Patterns – Machine Learning (Module 1)

Learning Objectives

By the end of this video you will:

1. **Define** unsupervised learning and how it differs from supervised learning.
 2. **Describe** two main types: **clustering** and **association rule mining**.
 3. **Explain** the goal of clustering (homogeneous groups) and association rules (item relationships).
 4. **Give** real-world applications of both.
-

Supervised vs Unsupervised (Quick Recap)

	Supervised	Unsupervised
Data	X and Y (labels)	Only X (no labels)
Goal	Prediction (of Y)	Discovery of structure, patterns, relationships
Also known as	Prediction methods	Descriptive methods / knowledge discovery

In unsupervised learning we **do not predict** a target; we **explore** the data to find **hidden structure and patterns**.

Two Primary Types of Unsupervised Learning

1. Clustering

- **Goal:** Automatically **group similar** data points together; put **dissimilar** points in different groups.
- **No labels:** We only have **X**. The algorithm finds “natural” groups (e.g. by language, by topic, by behaviour).
- **Idea:** Minimize **intra-cluster distance** (points in same group are close) and maximize **inter-cluster distance** (different groups are far). Equivalently: maximize similarity within a group, minimize similarity across groups.
- **Similarity** can be defined in many ways (e.g. language, height, features); the choice defines what “similar” means.

Example – Indian states (historical): States were formed so that “similar” people (e.g. same language) are in one state and “dissimilar” (different language) in different states. Same idea as

clustering: group by a notion of similarity (e.g. language).

Applications:

- **Google News:** Cluster news articles by **topic** (sports, finance, national, etc.).
- **Stock / market analysis:** Group companies (e.g. oil, tech, defence) that behave similarly.
- **Market segmentation:** Divide customers into segments for targeted marketing (e.g. high-income vs low-income for product launch).

2. Association Rule Mining

- **Goal:** Find **relationships / rules** between variables (e.g. "if item A is bought, then item B is often bought").
- **Focus:** Co-occurrence, dependency rules: "occurrence of one item predicts occurrence of another."
- **Example:** "If **samosa** is bought → **Coca-Cola** is often bought." Used for shelf placement, inventory, recommendations.

Applications:

- **Market basket / supermarket:** Products bought together → place them together; inventory and layout.
- **Recommendations:** "Customers who bought X also bought Y."

Clustering in More Detail

- **Input:** Set of data points (only **X**), no class labels.
- **Output:** Groups (clusters) such that:
 - Points in the **same** cluster are **similar**.
 - Points in **different** clusters are **dissimilar**.
- **Pure/homogeneous cluster:** Ideally (or approximately) all points in one cluster belong to one "type"; then we can interpret or label the cluster.
- **Impure/heterogeneous:** Mix of types; may need further splitting or a different number of clusters.

Summary Table

Unsupervised type	Goal	Example
Clustering	Find natural groups ; similar together, dissimilar apart	News by topic, customer segments, states by language
Association rule mining	Find rules like "if A then B" (frequent itemsets, dependencies)	Market basket, shelf layout, recommendations

Exam-Oriented Points

- Unsupervised = **no Y**; goal = **patterns/structure**, not prediction.
- **Clustering** = grouping by similarity; minimize within-group distance, maximize between-group.
- **Association rule mining** = dependency rules between items (e.g. purchase A → purchase B).
- Applications: document clustering, market segmentation, market basket, recommendations.

Use this note for definitions, difference from supervised learning, and application examples.

Practical Considerations in Machine Learning – Machine Learning (Module 1)

Learning Objectives

By the end of this video you will:

1. **List** real-world challenges when building ML systems.
 2. **Explain** why **scalability, dimensionality, data quality, heterogeneity, ownership, privacy, and streaming** matter.
 3. **Connect** these issues to building **effective and efficient** ML pipelines.
-

Building an Effective ML System: What to Consider

An ML system (classification, regression, clustering, association rules, etc.) should be:

- **Scalable**
- Able to handle **complex and heterogeneous** data
- Aware of **data ownership and distribution**
- Able to deal with **high-dimensional** data
- Robust to **poor data quality**
- **Privacy-preserving** and compliant

Below are the main **practical challenges** and why they matter.

1. Scalability

- **Meaning:** The algorithm and pipeline should work when data **grows** (e.g. from 100 to 100,000 or 1 billion records).
 - **Requirement:** Performance (time, memory) should remain acceptable as data size increases. The system must **scale** with data volume.
 - **Why it matters:** In production, data often grows; a solution that only works on small samples is not useful.
-

2. Dimensionality (High-Dimensional Data)

- **Meaning: Dimensionality** = number of **features/attributes** (e.g. number of columns in a table).

- **Problem:** Many ML algorithms perform **poorly** or become **slow** when the number of dimensions is large. The amount of data needed to support reliable results often grows with dimensions.
 - **Sparsity:** In high dimensions, data can become **sparse** (few points per region). That makes learning harder.
 - **Requirement:** The pipeline should handle high-dimensional and possibly sparse data (e.g. via feature selection, dimensionality reduction, or algorithms suited to high dimensions).
-

3. Heterogeneous and Complex Data

- **Meaning:** Data comes from **many sources** and in **many forms**: audio, video, text, sensors, databases.
 - **Challenge:** We must **aggregate and integrate** these sources into a form suitable for modeling. Data can also be **complex** in structure (e.g. nested, graph, time series).
 - **Requirement:** The ML pipeline must **integrate and handle** heterogeneous and complex data effectively.
-

4. Data Quality

- **Reality:** Real-world data is often **poor quality**: noisy, incomplete, inconsistent, with outliers and missing values.
 - **Principle:** **Garbage in, garbage out.** Model quality depends heavily on input data quality.
 - **Practice:** A large part of an ML project (often quoted ~80%) is **data cleaning and preparation**.
 - **Requirement:** The pipeline must handle **noise, missing values, outliers, inconsistencies** (e.g. through data cleaning, imputation, robust methods).
-

5. Data Ownership and Distribution

- **Meaning:** In large organizations, data lives in **different departments/silos**. Ownership and sharing can be unclear or politically sensitive.
 - **Challenge:** Building a **unified view** for ML may require integrating data across owners; this can create ownership and governance issues.
 - **Requirement:** Data integration and access must respect **ownership** and **distribution** so that the project is technically and politically feasible.
-

6. Privacy

- **Problem:** Many datasets contain **sensitive or personal** information. Leaks or misuse cause harm and **legal** issues (e.g. GDPR).

- **Requirement:** Data must be **stored and processed** in an **ethical and legally compliant** way. The pipeline should support **privacy preservation** (e.g. anonymization, access control, secure processing).
-

7. Streaming and Diverse Data Types

- **Reality:** Data may arrive as **streams** (e.g. sensors, social media, finance) rather than as a single static batch.
 - **Challenge:** Traditional batch-only systems may not handle **real-time** or **continuously flowing** data.
 - **Requirement:** The pipeline should be able to handle **streaming data**, as well as **static, dynamic, image, video**, etc., as needed by the application.
-

Module 1 Recap (Four Videos)

Video	Focus
1	Foundations: What is ML, why ML, pipeline, workflows, data types.
2	Supervised learning: Classification and regression; training and testing.
3	Unsupervised learning: Clustering and association rule mining; finding patterns.
4	Practical considerations: Scalability, dimensionality, quality, heterogeneity, ownership, privacy, streaming.

Summary

- **Scalability:** System must handle growing data.
- **Dimensionality:** High dimensions and sparsity make learning harder; pipeline must account for this.
- **Heterogeneous/complex data:** Multiple sources and formats require integration and appropriate handling.
- **Data quality:** Poor quality → poor models; invest in cleaning and preparation.
- **Ownership & distribution:** Integration must respect organizational and data ownership.
- **Privacy:** Ethical and legal handling of sensitive data.
- **Streaming:** Support for real-time and varied data types when required.

Use this note for exam questions on “challenges in ML projects” and “what to consider when building an ML system.”

Week 2

Building Blocks of Data – Objects, Attributes and Types – Machine Learning (Module 2)

Learning Objectives

By the end of this video you will:

1. **Define** dataset, data objects, and the role of **attributes**.
 2. **Distinguish** rows (tuples/records) vs columns (attributes/features).
 3. **Classify** attributes as **nominal**, **ordinal**, **interval**, and **ratio**.
 4. **State** which **mathematical operations** are valid for each attribute type.
 5. **Define discrete** vs **continuous** attributes.
-

Dataset and Data Objects

- **Dataset:** A collection of data objects stored in a **structured** way (e.g. table).
- **Why structure?** Large data (e.g. millions of transactions) must be stored and retrieved efficiently; structure enables that.
- **Examples:** Bank passbook (date, transaction ID, credit, debit, balance), sales DB, medical DB, university DB.

In a **table**:

- **Rows** = data points = **records / tuples / cases / samples / entities / instances**.
 - **Columns** = **attributes** (or **features**). Each attribute describes a **property or characteristic** of the data object.
 - A **single object** is described by the **collection of attribute values** in one row.
-

Attributes and Attribute Values

- **Attribute:** A property or characteristic of a data object. Synonyms: attribute, feature.
 - **Attribute value:** The specific number or symbol assigned to that attribute for a given object (e.g. Refund = Yes, Marital status = Single, Taxable income = 125).
 - Different attributes can have different **numbers of unique values** (e.g. Refund: 2; Marital status: 3; Taxable income: many/numerical).
-

Two Broad Categories of Attributes

Category	Also known as	Measures	Sub-types
Qualitative	Categorical	Quality / category	Nominal, Ordinal
Quantitative	Numerical	Quantity / amount	Interval, Ratio

The Four Attribute Types (Detail)

1. Nominal

- **Meaning:** Values are **labels or names** used only to **distinguish** one object from another. No order, no magnitude.
- **Operations: Equality only** — equal to ($=$) and not equal to ($\neq$).
- **Examples:** ID number, colour (blue, green), zip code. We can only say two values are same or different.

2. Ordinal

- **Meaning:** Values can be **ranked or ordered** (ascending/descending). **Distance between values has no meaning** (e.g. "how much better" is not defined).
- **Operations: Equality + order** — $=, \neq, <, >, \leq, \geq$.
- **Examples:** Grade (A, B, C, D, F), rank (1, 2, 3), height (tall, medium, short).

3. Interval

- **Meaning:** **Differences** between values **have meaning** and can be measured in a **fixed unit**. No true zero for ratios (e.g. 0°C is not "no temperature").
- **Operations: Equality + order + addition/subtraction** — $=, \neq, <, >, +, -$.
- **Examples:** Calendar date, temperature (Celsius/Fahrenheit). e.g. "Date 2 – Date 1 = 1 day" is meaningful.

4. Ratio

- **Meaning:** Has all properties of **interval** plus a **true zero**; **ratios** of values are meaningful (e.g. "twice as long").
- **Operations: Equality + order + addition/subtraction + multiplication/division** — $=, \neq, <, >, +, -, \times, \div$.
- **Examples:** Length, time, count, distance (e.g. $1000\text{ km} \times 10 = 10,000\text{ km total}$).

Summary: Operations by Type

Type	Distinctness (=, ≠)	Order (<, >)	Addition/subtraction	Multiplication/division
Nominal	✓	×	×	×
Ordinal	✓	✓	×	×
Interval	✓	✓	✓	×
Ratio	✓	✓	✓	✓

Discrete vs Continuous Attributes

Type	Meaning	Example
Discrete	Finite or countable number of distinct values	Zip code, binary (0/1), grade (A–F)
Continuous	Infinite possible values (e.g. real numbers)	Temperature (10, 10.1, 10.11, ...), age, weight

- **Discrete:** Cardinality is finite or countable.
 - **Continuous:** Values can be arbitrarily fine (e.g. 10.111...).
-

Exam-Oriented Points

- **Rows** = tuples/records; **columns** = attributes/features.
- **Nominal:** only same/different; **ordinal:** + order; **interval:** + difference; **ratio:** + ratio and true zero.
- **Discrete** = finite/countable values; **continuous** = infinite possible values.
- Always ask: "Which operations are valid?" for a given attribute type.

Use this note for definitions and for choosing correct operations and attribute types in exam questions.

Exploring Different Dataset Formats – Machine Learning (Module 2)

Learning Objectives

By the end of this video you will:

1. **Distinguish** three main **data organization** types: **record**, **graph**, and **ordered** data.
 2. **Describe record data** and its sub-types: **data matrix**, **document data**, **transactional data**.
 3. **Describe graph data** (nodes, edges) and give examples (e.g. web, molecules).
 4. **Describe ordered/sequence** data (order matters) and **spatio-temporal** data.
 5. **Explain** why **structure** determines which **questions** and **analysis tools** we can use.
-

Three Main Ways to Organize Data

After we **structure** raw information for storage and analysis, we get different **dataset formats**:

Type	Organization	Typical use
Record data	Table-like: rows = objects, columns = attributes	Most common; many ML algorithms
Graph data	Nodes + edges (relationships)	Networks, links, dependencies
Ordered data	Sequence where order matters	Time series, genomes, events

1. Record Data

- **Definition:** A **collection of records** (data objects), each with a **fixed set of attributes**.
- **Tabular data** is the standard example: rows = records, columns = attributes.

Record data can be further categorized:

Data Matrix

- **All attributes are numerical.**
- Can be thought of as an **m × n matrix**; each object is a point in an **n-dimensional** space.
- Used when we have only numeric features (e.g. for distance-based or linear algebra methods).

Document Data

- **Objects = text documents** (e.g. news articles).
- Each document is often represented as a **term vector**: **attributes = distinct terms (words)** in the collection; value = count (or weight) of that term in the document.
- **Result**: Often a **sparse** matrix (most documents do not contain most terms).

Transactional Data

- Each **record = one transaction** (e.g. one market basket).
 - **Attributes are not fixed** in the same way: each record is a **set of items** that appear together (e.g. {bread, milk, coke}).
 - **Example**: Supermarket: TID (transaction ID) + list of items bought. Used for **association rule mining** (e.g. "if bread and butter, then milk").
-

2. Graph Data

- **When to use**: When **relationships between objects** are central; information is not only in the attributes of single objects but in **links** between them.
 - **Structure**: **Nodes** = objects; **edges** = connections/relationships.
 - **Examples**:
 - **World Wide Web**: Nodes = web pages, edges = hyperlinks. Used in **PageRank** and crawling.
 - **Molecules**: Nodes = atoms, edges = chemical bonds.
 - **Analysis**: Graph algorithms (e.g. centrality, community detection, path finding).
-

3. Ordered Data

- **Definition**: Data where **order** of objects **matters** (e.g. first event, second event, ...).
 - **Examples**:
 - **Genome sequence** — sequence of bases (A, T, G, C).
 - **Time series** — values ordered by time.
 - **Spatio-temporal** — data with both **space** and **time** (e.g. location at each timestamp).
-

Why Structure Matters

- The **structure** of the dataset determines:
 - **What questions** we can ask (e.g. "which items are bought together?" → transactional; "which pages link to this?" → graph).
 - **What analysis tools** we can apply (e.g. clustering on data matrix vs graph algorithms on graph data).

- **Choosing the right representation** is a **critical step** in a data pre-processing / ML project.

Summary Table

Format	Main idea	Sub-types / examples
Record	Table: rows = objects, columns = attributes	Data matrix (all numeric), document (term vectors), transactional (item sets)
Graph	Nodes + edges	Web (pages + links), molecules (atoms + bonds)
Ordered	Order matters	Sequences, time series, spatio-temporal

Exam-Oriented Points

- **Record:** Tabular; data matrix (numerical), document (terms as attributes, sparse), transactional (sets of items).
- **Graph:** Nodes and edges; relationships are first-class (e.g. web, social network, molecules).
- **Ordered:** Sequence or time order matters (genome, time series, spatio-temporal).
- **Structure** drives **questions** and **tools** we can use.

Use this note for questions on "types of dataset formats" and "when to use record vs graph vs ordered data."

Data Quality Issues – Machine Learning (Module 2)

Learning Objectives

By the end of this video you will:

1. **Define** data quality and why it matters for ML.
 2. **List** key **dimensions** to measure data quality.
 3. **Describe** common data quality **problems**: noise, outliers, missing values, duplicates, inconsistency.
 4. **Explain** the principle “garbage in, garbage out.”
 5. **State** the role of **data pre-processing** as a solution.
-

Why Data Quality Matters

- **Ideal**: Data is perfect, complete, and ready for analysis.
 - **Reality**: Raw data is often **messy, incomplete, and inconsistent**.
 - **Principle: Garbage in, garbage out (GIGO)**. If we build a model (prediction or descriptive) on **low-quality data**, the results will be **low-quality or unreliable**.
-

Definition of Data Quality

- **Data quality** = the **overall utility** of the dataset and its **fitness for the intended purpose** (e.g. building a credit-card fraud detector, answering business questions).
 - **Golden rule**: Data has quality **if it satisfies the requirements of the intended user**.
 - **High-quality data** correctly represents the real-world construct it is meant to represent.
 - **Subjective**: The same dataset can be “high quality” for one user (e.g. R&D) and “low quality” for another (e.g. delivery) depending on the questions they ask.
-

Dimensions of Data Quality

Data quality is **multi-dimensional**. Key dimensions include:

Dimension	Meaning
Accuracy	Is the data correct ? (e.g. names spelled correctly, values not wrong.)
Completeness	Is all necessary information present? (e.g. address fully filled.)
Consistency	Does the data contradict itself? (e.g. age vs date of birth mismatch.)
Timeliness	Is the data up to date ? (e.g. current address.)
Believability	How much can we trust the data?
Interpretability	Can we understand the data easily?

These dimensions give a **framework** to **measure** and **improve** data quality.

Common Data Quality Problems

1. Noisy Data

- **Noise** = random **error or variance** in a measured variable; a **modification** of the true value.
- **Causes**: Faulty sensors, transmission errors, **data entry mistakes** (e.g. census, digitization), intentional misreporting.
- **Effect**: Distorts analysis and models. Like static on a phone call — the true signal is corrupted.

2. Outliers

- **Outliers** = data objects that are **legitimate** (real values) but **very different** from most of the other objects.
- **Not the same as noise**: Noise is **error**; outliers are **extreme but true** values.
- **Two roles**:
 - **Nuisance**: They can **distort** statistics (e.g. average) and models. Example: values 1, 2, 3, 4, 100 → average 22; without outlier, average 2.5.
 - **Goal**: Sometimes **finding** outliers is the objective (e.g. **credit card fraud, intrusion detection** — the “bad” cases are rare and outlier-like).

3. Missing Values

- Some **values are absent** (empty fields).
- **Causes**: Information not collected, attribute not applicable, equipment failure, human oversight.
- **Effect**: Incomplete records; many algorithms cannot handle missing values directly — we must **handle** them (e.g. impute, remove, or model).

4. Duplicate Data

- Two (or more) **records or attributes** are **same or nearly same**.

- **Common when:** Merging data from **multiple sources** without careful deduplication.
- **Effect:** Redundancy, biased counts, and wasted storage; can distort statistics and models.

5. Inconsistent Data

- Data **contradicts** itself (e.g. age 25 but date of birth implies 30).
 - **Effect:** Unreliable conclusions; we need rules and checks to enforce consistency.
-

Real-World Reality and Pre-Processing

- In practice, data is often **incomplete, noisy, inconsistent**, and may contain **outliers** and **duplicates**.
 - **Consequence:** Bad business decisions, loss of trust, waste of time and resources.
 - **Solution: Data pre-processing** — a set of steps to **improve** quality and **prepare** data for the next stages (e.g. modeling).
 - **Goals of pre-processing:**
 - i. **Improve quality** — handle noise, missing values, outliers, inconsistency, duplicates.
 - ii. **Make data fit the model** — structure and transform data so it is suitable for the chosen ML/data mining algorithm.
-

Summary

- **Data quality** = fitness for intended use; measured by accuracy, completeness, consistency, timeliness, believability, interpretability.
- **Common problems:** Noise (error), outliers (extreme but real), missing values, duplicates, inconsistency.
- **GIGO:** Poor data → poor model and poor decisions.
- **Pre-processing** is the main way we address quality issues before building models.

Use this note for exam questions on "what is data quality," "dimensions of quality," and "common data quality problems."

Data Preprocessing Techniques – Machine Learning (Module 2)

Learning Objectives

By the end of this video you will:

1. **List** the four major **tasks** of data pre-processing: cleaning, integration, transformation, reduction.
 2. **Describe** how to handle **missing values** and **noisy data**.
 3. **Explain data integration** issues (e.g. entity identification, value conflicts, redundancy).
 4. **Describe normalization** (min-max, z-score) and **aggregation**.
 5. **Explain data reduction** (by attributes and by number of records).
-

Why Pre-Processing?

- Real-world data is **messy**; **garbage in, garbage out**.
 - Pre-processing is **not one step** but a **set of tasks** to improve quality and prepare data for ML/data mining.
-

Four Major Categories of Pre-Processing Tasks

Task	Purpose
Data cleaning	Fix missing values, smooth noise, identify/handle outliers, resolve inconsistency.
Data integration	Merge data from multiple sources; handle conflicts and redundancy.
Data transformation	Change scale/structure (e.g. normalization, aggregation) for algorithms.
Data reduction	Reduce volume (rows/columns or both) while preserving integrity.

1. Data Cleaning

Handling Missing Values

- **Ignore the tuple** — drop records with missing values (simple but can lose data).
- **Fill manually** — often tedious or infeasible for large data.
- **Global constant** — replace missing with a constant (e.g. "Unknown," "ABC").
- **Central tendency** — for numerical attributes use **mean**, **median**, or **mode** of the attribute.
- **Conditional / model-based** — use rules (e.g. if A = 1 then fill B with 2) or a model to impute.

Handling Noisy Data

- **Smoothing** — reduce impact of noise:
 - **Binning**: Sort data, partition into **bins**, then replace values in each bin by **bin mean**, **median**, or **boundary** (local smoothing).
 - **Regression**: Fit a regression; use fitted values as smoothed values.
 - **Outlier/cluster analysis**: Detect points outside clusters as potential noise and handle separately.
-

2. Data Integration

- **When**: Merging data from **multiple sources** (e.g. different DBs, departments).
 - **Challenges**:
 - **Entity identification** — same real-world entity represented differently (e.g. multiple IDs for one person).
 - **Data value conflicts** — same entity with different values (e.g. "John Smith" vs "J.D. Smith," "Hemant R" vs "H Rathaur" on different ID cards).
 - **Redundancy** — duplicate or overlapping attributes/tuples after merge.
 - **Example (Aadhaar)**: Government needed a single view of citizens across voter ID, passport, ration card, etc. Merging led to serious **integrity** issues; eventually a **new**, dedicated data collection (Aadhaar) was used. Integration is **tedious and error-prone**.
-

3. Data Transformation

- **Goal**: Convert data into a **form suitable** for mining (e.g. scale, structure), **not** primarily to fix errors.
- **Common methods**:

Normalization

- **Why**: Avoid one attribute (e.g. income in lakhs) **dominating** distance-based or gradient-based calculations; often **speeds up** training.
- **Min-max normalization**: Map original range [min, max] to a new range [new_min, new_max]:
 - **Formula**: $v' = (v - \min) / (\max - \min) * (\text{new_max} - \text{new_min}) + \text{new_min}$
 - Preserves **shape** of the distribution in the new range.

- **Z-score normalization:** Use mean and standard deviation:
 - **Formula:** $v' = (v - \text{mean}) / \text{std_dev}$
 - Result is in units of standard deviations from the mean.

Aggregation

- **Combine** values to change **scale** or **granularity** (e.g. city → state → country; quarterly → yearly).
 - **Benefits:** Data can become **more stable** and **less variable**; also **reduces** volume.
-

4. Data Reduction

- **Goal:** Obtain a **smaller** (reduced) dataset that is **representative** and **preserves** the integrity/properties of the original, so that analysis and mining are **faster** and more feasible.

Reduction by Attributes (Dimensionality Reduction)

- **Idea:** Use fewer attributes (drop or combine).
 - **Irrelevant attributes** — not useful for the analysis (e.g. student ID for predicting GPA).
 - **Redundant attributes** — carry (almost) the same information; one can be removed.
- **Methods:** Attribute **subset selection** (e.g. forward selection, backward elimination), decision-tree-based selection, other feature selection techniques.

Reduction by Number of Records

- **Sampling** — use a **representative sample** (e.g. simple random, stratified) so the sample behaves like the full data.
 - **Histograms, clustering, aggregation** — summarize or group records to reduce count while preserving structure.
-

End-to-End View

1. Start with **raw, messy** data.
 2. **Clean** (missing values, noise, outliers, inconsistency).
 3. **Integrate** carefully (entity ID, value conflicts, redundancy).
 4. **Transform** (normalize, aggregate) for the algorithm.
 5. **Reduce** (attributes and/or records) to a manageable, representative set.
 6. Result: **higher-quality** data ready for **modeling**.
-

Summary

- **Four task groups:** Cleaning, integration, transformation, reduction.
- **Cleaning:** Missing value strategies (ignore, constant, mean/median, model) and noise (binning, regression, outlier handling).
- **Integration:** Beware of entity identification, value conflicts, redundancy.
- **Transformation:** Normalization (min-max, z-score), aggregation.
- **Reduction:** Fewer attributes (subset selection), fewer records (sampling, histograms, clustering).

Use this note for exam questions on "steps of pre-processing" and "how to handle missing values and noise."

Week 3

Foundations of Machine Learning and Supervised Learning – Machine Learning (Module 3)

Learning Objectives

By the end of this video you will:

1. **Define** Machine Learning and its fundamental purpose (learn from data without explicit rules).
 2. **Differentiate supervised** vs **unsupervised** (and briefly **reinforcement**) learning.
 3. **Explain** core concepts of supervised learning: X , Y , model f , prediction.
 4. **Distinguish classification** (discrete Y) vs **regression** (continuous Y).
 5. **Describe** the end-to-end ML workflow (problem $\rightarrow$ data $\rightarrow$ preprocess $\rightarrow$ model $\rightarrow$ evaluate $\rightarrow$ deploy).
-

What Is Machine Learning?

Traditional approach (e.g. teaching a child to recognize a cat): We give **rules** — “four legs, pointy ears, whiskers, meows = cat.” The learner follows explicit instructions.

ML approach: We give **many examples** (e.g. thousands of cat images) and let the **algorithm learn** patterns and features that associate images with “cat” **without** programming every rule.

Definition: Machine Learning is the **science of getting computers to learn and act** (e.g. like humans) **by learning patterns from data without being explicitly programmed** with rules.

Traditional Programming vs Machine Learning

Traditional	Machine Learning
Input + Rules $\rightarrow$ Output	Data + Problem statement $\rightarrow$ Model $\rightarrow$ Output (predictions)
We write the rules	Algorithm learns rules/patterns from data

ML workflow (high-level):

1. **Problem** — e.g. “Distinguish cat vs dog in images.”
2. **Collect data** — e.g. thousands of images of cats and dogs.
3. **Pre-process** — clean and prepare data.
4. **Build model** — choose an algorithm (e.g. decision tree, random forest, SVM), **train** on data to get model f .

5. **Evaluate** — measure performance (e.g. on new images).

6. **Deploy** — use **f** in the real world for prediction.

We **do not** tell the algorithm exactly how to separate cat and dog; it **learns** from data.

Three Types of Learning

Type	Idea	Goal
Supervised	Learn with a “teacher” — we have labels (Y)	Prediction (of Y for new X)
Unsupervised	No labels; only X	Find patterns and structure in data
Reinforcement	Learn by trial and error with rewards/penalties	Learn actions that maximize reward

This course focuses on **supervised** and **unsupervised** learning.

Supervised Learning in Detail

- **Input:** Data **X** (e.g. attributes) and **labels Y** (correct answers / class labels).
- **Supervisor** = training data **with** labels (the “answer key”).
- **Goal:** Learn a **function f** such that **$f(X) \approx Y$** . For **new/unseen X**, we **predict Y** using **f**.

Example – Spam filter:

- **X:** Email content, sender, attachments, etc.
- **Y:** Spam or Not spam.
- We build **f** from many (X, Y) examples. For a new email (X only), **f** predicts spam or not.

Example – Unsupervised (contrast):

- Only **X** (e.g. customer purchase data). No “spam/not spam.”
- Goal: **Find patterns** — e.g. “grocery shoppers,” “tech buyers,” “new parents” — **no prediction** of a label.

Classification vs Regression (Supervised)

Both use **X** and **Y** to learn **f** and then **predict Y** for new **X**.

	Classification	Regression
Y	Discrete / categorical (finite classes)	Continuous / numerical
Examples	Spam/not spam, malignant/benign, cat/dog/bird	Petrol price, salary, stopping distance
Number of values	Fixed (e.g. 2, 3)	Infinite (any number in range)

Classification: Separate **classes** (e.g. by a line/plane/curve). New point → assign one of the predefined classes.

Regression: Find a **relationship** between X and Y (e.g. age vs salary). New X → predict a **numeric** Y.

Visual Intuition

- **Classification (binary):** Two classes (e.g. squares and circles). We learn a **boundary**; points on one side → class 1, other side → class 2. **Multi-class:** More than two classes; boundary separates all.
- **Regression:** Scatter of (X, Y) (e.g. age vs salary). We fit a **curve/line**; for new X we **predict** Y on that curve.

Summary

- **ML** = learning patterns from data **without** explicit programming of rules.
- **Supervised:** Have **X** and **Y**; learn **f** to **predict Y** for new **X**.
- **Unsupervised:** Only **X**; find **patterns/structure** (no prediction of a label).
- **Classification:** Y = discrete/categorical. **Regression:** Y = continuous.
- **Workflow:** Problem → Data → Preprocess → Train model → Evaluate → Deploy.

Use this note for definitions and for comparing supervised vs unsupervised and classification vs regression in exams.

The Core Principles of Classification – Machine Learning (Module 3)

Learning Objectives

By the end of this video you will:

1. **Define** the classification task formally (training data, attribute set X , class label Y , model f).
 2. **State** that class labels must be **mutually exclusive** and **exhaustive**.
 3. **Describe** the **two-step** process: **model training** and **model testing**.
 4. **Explain** why test set must be **independent** of training set.
 5. **Give** real-world applications (e.g. spam, tumor, malware).
-

Formal Definition of Classification

- **Training data:** A collection of **records**; each record has X (attribute set) and Y (class label).
- **X (attribute set):** Set of **features** (e.g. x_1, x_2, x_3) that **describe** each record.
- **Y (class label):** **Predefined category** the record belongs to (what we want to predict).
- **Task:** Learn a **model (function)** f that **maps** $X \rightarrow Y$ so that for **new/unseen** records we can **assign a class label** as accurately as possible.

Example – Customer churn:

- **X :** Age, billing amount, months on platform, number of support calls.
 - **Y :** Stay or Unsubscribe (two classes).
 - **Goal:** Given a new customer's X , predict whether they will **stay** or **unsubscribe**.
-

Properties of Class Labels (Y)

- **Exhaustive:** Every possible outcome fits into **one** of the classes (we have a label for every record we care about).
 - **Mutually exclusive:** No object belongs to **more than one** class. If two records have the **same X** , they must have the **same Y** (no contradiction).
-

Real-World Examples

Application	X (examples)	Y (classes)
Email categorization	Words, sender, attachment, etc.	Spam, Not spam
Tumor / cancer	Size, shape, texture, boundary (from image)	Malignant, Benign
Malware detector	File features (a1, a2, a3, ...)	Malware (M), Benign (B)

In each case we have **training** (X, Y); we build **f**; for **new X** we **predict Y**.

Two-Step Process: Training and Testing

Step 1: Model Training

- **Input: Training set** (subset of full data) with **X** and **Y**.
- **Process:** Use a **learning algorithm** (e.g. decision tree, rules, SVM) to **learn** the mapping from **X** to **Y**.
- **Output: Model f** (e.g. a set of rules or a tree). Example rule: "If age = 60 and has_loan = true → class = high."

Step 2: Model Testing

- **Input: Test set** (different from training set). We have **X** and **Y** but **temporarily ignore Y**.
- **Process:** For each test record, feed **only X** into **f**; get **predicted Y** (e.g. $\hat{y}$). Compare $\hat{y}$ with **true Y**.
- **Goal: Estimate** how well **f** will perform on **unseen** data. If predictions match true labels often → good model; else → need improvement.

Critical: Test set must be independent of training set (no overlap). Evaluating on training data would be "cheating" (like giving exam questions that are identical to practice). We want to test **generalization** to new data.

Deployment (Inference)

- Once we are satisfied with **test performance**, we **deploy** the model.
 - In production we only get **X** (e.g. new email, new image, new file). We use **f** to **predict Y** (spam/not, malignant/benign, malware/benign).
 - This use of the **built** model for prediction is sometimes called **deduction** or **inference**.
-

Full Pipeline (Recap)

1. Start with **full labeled data** (X, Y).
 2. **Split** into **training** and **test** (disjoint).
 3. **Train** model f on **training** data.
 4. **Test** f on **test** data (feed X , compare predicted Y with true Y).
 5. If performance is good $\rightarrow$ **deploy** f for real-world prediction.
-

Summary

- **Classification** = learn $f: X \rightarrow Y$ from (X, Y) , with Y categorical; **exhaustive** and **mutually exclusive**.
- **Training** = build f using **training set**; **testing** = evaluate f on **separate test set** (only X fed to f ; compare predictions to true Y).
- **Test set** must be **independent** of training to measure true performance.
- **Deployment** = use f to predict Y for new X in the real world.

Use this note for exam questions on "formal definition of classification," "training vs testing," and "why test set must be separate."

Evaluating Model Performance and Common Pitfalls – Machine Learning (Module 3)

Learning Objectives

By the end of this video you will:

1. **Define** train error vs test error and how they are computed.
 2. **Build** a **confusion matrix** (TP, TN, FP, FN) for binary classification.
 3. **Define** and compute **accuracy**, **precision (PPV)**, **recall (TPR/sensitivity)**, **NPV**, **TNR (selectivity)**.
 4. **Describe** other evaluation aspects: **speed** (training/prediction), **interpretability**, **scalability**, **robustness**.
 5. **Compare** evaluation for **classification** vs **regression** (correct/incorrect vs error magnitude).
-

Train Error vs Test Error

- **Train error:** Train the model on **training data**, then **evaluate on the same training data**. Measures how well the model **fits** the training set.
- **Test error:** Train the model on **training data**, then **evaluate on held-out test data**. Measures how well the model **generalizes** to unseen data.

We **care most** about **test** performance for real-world use.

Evaluating a Classification Model (Correct vs Incorrect)

- For each **test** record we have **true label** Y and **predicted label** $\hat{y}$.
 - **Correct prediction:** $\hat{y} = Y$. **Incorrect prediction:** $\hat{y} \neq Y$.
 - We **count** correct and incorrect over all test samples. These counts underlie **accuracy** and the **confusion matrix**.
-

Confusion Matrix (Binary Classification)

Assume two classes: **Positive** (e.g. Malware) and **Negative** (e.g. Benign). Rows = **true** class, columns = **predicted** class.

	Predicted Positive	Predicted Negative
True Positive	TP (correct)	FN (wrong)
True Negative	FP (wrong)	TN (correct)

- **TP:** True class = Positive, Predicted = Positive. ✓
- **TN:** True class = Negative, Predicted = Negative. ✓
- **FP:** True class = Negative, Predicted = Positive. ✗ (false alarm)
- **FN:** True class = Positive, Predicted = Negative. ✗ (miss)

Correct predictions = TP + TN. **Incorrect** = FP + FN.

Accuracy

- **Accuracy** = (Correct predictions) / (Total predictions) = $(TP + TN) / (TP + TN + FP + FN)$.
 - Often expressed as a percentage ($\times 100$). Higher accuracy = better overall correctness.
-

Precision (PPV – Positive Predictive Value)

- **Precision** = Of all instances predicted as Positive, how many were **actually** Positive?
 - **Formula:** Precision = $TP / (TP + FP)$.
 - **Focus:** How reliable are our positive predictions?
-

Recall (TPR – True Positive Rate / Sensitivity)

- **Recall** = Of all instances that are **actually** Positive, how many did we **correctly** predict as Positive?
 - **Formula:** Recall = $TP / (TP + FN)$.
 - **Focus:** How many of the true positives did we catch?
-

NPV (Negative Predictive Value)

- Of all instances predicted as Negative, how many were **actually** Negative?
 - **Formula:** NPV = $TN / (TN + FP)$.
-

TNR (True Negative Rate / Selectivity)

- Of **all** instances that are **actually Negative**, how many did we **correctly** predict as Negative?
- Formula: $TNR = TN / (TN + FP)$.

Train/Test Split (Typical Ratios)

- Common: **70–30** or **80–20** (train–test). Also **90–10**, **60–40** in literature. **No fixed rule**; data and problem dictate.
 - Idea: Most data for **training**, a separate portion for **testing** only.
-

Evaluating a Regression Model

- For regression, Y and $\hat{y}$ are **continuous**. We cannot just count “correct/incorrect.”
 - **Error** for one sample: $\hat{y} - Y$ (or $|\hat{y} - Y|$).
 - **Aggregate measures**:
 - **Sum of absolute errors**: $\sum |\hat{y} - Y|$ over test samples.
 - **Mean absolute error (MAE)**: $(1/n) \sum |\hat{y} - Y|$.
 - **Sum of squared errors (SSE)**: $\sum (\hat{y} - Y)^2$.
 - **Mean squared error (MSE)**: $(1/n) \sum (\hat{y} - Y)^2$.
-

Other Performance Criteria

Criterion	Meaning
Speed (training)	Time to train the model (seconds to days).
Speed (prediction)	Time to predict one (or many) sample(s). Critical in real-time systems (e.g. intrusion detection).
Interpretability	Can we explain how the model reached a decision? High = white box; low = black box (e.g. complex neural nets).
Scalability	Does performance (time, memory) remain acceptable as data size or load grows?
Robustness	Does the model handle noise and outliers reasonably?

Summary

- **Train error** = error on training data; **test error** = error on held-out test data.
- **Confusion matrix**: TP, TN, FP, FN. **Accuracy** = $(TP + TN) / \text{total}$.

- **Precision** = $TP/(TP+FP)$; **Recall** = $TP/(TP+FN)$. PPV/NPV and TPR/TNR give a full picture.
- **Regression**: Use **absolute** or **squared** errors (e.g. MAE, MSE).
- Also consider **speed**, **interpretability**, **scalability**, **robustness** for real systems.

Use this note for computing and interpreting confusion matrix, accuracy, precision, and recall in exams.

Common Pitfalls – Overfitting and Underfitting – Machine Learning (Module 3)

Learning Objectives

By the end of this video you will:

1. **Define underfitting** and **overfitting** in terms of model complexity, train error, and test error.
 2. **Interpret** the **complexity–error** curve (train vs test error vs model complexity).
 3. **Explain** the **goal of generalization** and the “sweet spot” for model complexity.
 4. **Relate** overfitting to **learning noise** and **rote learning**; underfitting to **too-simple** models.
 5. **Describe** the **binary classification** picture: good decision boundary vs overfit (fitting noisy points).
-

Two Types of Error

- **Train error:** Error when we **train** on training data and **evaluate on the same** training data.
- **Test error:** Error when we **train** on training data and **evaluate on separate test** data.

We **plot** both against **model complexity** (e.g. tree depth, number of parameters).

Model Complexity vs Error (Typical Picture)

- **X-axis:** Model complexity (low → high).
- **Y-axis:** Error.

Train error: As complexity **increases**, train error generally **decreases** and can go very low (model can fit training data very well, even memorize it).

Test error: As complexity **increases**, test error first **decreases** then **increases**. So there is a **sweet spot** in the middle.

Underfitting

- **When:** Model is **too simple** for the data (low complexity).
- **Symptoms:** **High train error** and **high test error**.
- **Reason:** Model **cannot capture** the underlying pattern in the data (e.g. linear model for a nonlinear problem).

- **Result:** Poor performance everywhere; the model is **not useful**.
-

Overfitting

- **When:** Model is **too complex**; it has **over-learned** the training data, including **noise** and **outliers**.
- **Symptoms:** Low train error but **high test error**.
- **Reason:** Model **memorizes** training examples (rote learning) instead of learning **general** patterns. On **new** (test) data it performs poorly.
- **Result:** Looks great on training data, **fails** on unseen data. **Dangerous** in production.

Analogy: Student memorizes exact practice exam answers; in the real exam (slightly different questions), the student fails. Memorization $\neq$ generalization.

Good Fit (Sweet Spot)

- **Where:** Moderate complexity where **train and test error** are **both relatively low** and **close to each other**.
 - **Meaning:** Model has learned **general** patterns, not noise. It will perform **similarly** on new data (good **generalization**).
-

Generalization (Formal Idea)

- **Generalization** = the model's ability to **learn** from training data and make **accurate predictions** on **new, unseen** samples drawn from the **same** (or similar) distribution.
 - We want the model to **adapt** to the true underlying pattern, not to the exact training set. A **good** model **generalizes** well (low test error when train and test are from the same distribution).
-

Visual (Binary Classification)

- **Good fit:** Decision boundary **separates** the two classes in a **smooth, sensible** way; a few **noisy** points may be on the "wrong" side, and we **do not** bend the boundary to fit every such point.
 - **Overfitting:** Decision boundary is **twisted** so that it **correctly** classifies (almost) every training point, including **noisy** ones. The boundary becomes **too specific** to the training set and will **misclassify** many test points.
-

Bias–Variance Intuition (Optional)

- **Underfitting** → **high bias** (model is too rigid/simple).
- **Overfitting** → **high variance** (model is too sensitive to the training set; changing data changes the model a lot).
- **Sweet spot** → **balanced** bias and variance.

Summary Table

	Underfitting	Good fit	Overfitting
Complexity	Too low	Moderate	Too high
Train error	High	Low	Very low
Test error	High	Low	High
Generalization	Poor	Good	Poor

Exam-Oriented Points

- **Underfitting:** High train + high test error; model too simple.
- **Overfitting:** Low train + high test error; model memorizes training data (noise, outliers).
- **Goal: Generalization** — good performance on **unseen** data; aim for the **sweet spot** in complexity.
- **Good model** = one that **generalizes** well; use an **independent test set** to measure that.

Use this note for exam questions on “what is overfitting/underfitting,” “how do train and test error behave with complexity,” and “what is generalization.”

Week 4

Introduction to Decision Trees – Machine Learning (Module 4)

Learning Objectives

By the end of this video you will:

1. **Define** a decision tree and its **role in classification**.
 2. **Identify** core components: **root node**, **internal/decision nodes**, **branches/edges**, **leaf nodes**.
 3. **Distinguish tree induction** (building the tree) vs **tree deduction** (using the tree for prediction).
 4. **Explain** why decision trees are **white-box** and **easy to set up**.
 5. **Trace** a simple example: tax-evasion (refund, marital status, taxable income) → approve/reject.
-

Where Decision Trees Fit

- ML → **Supervised** → **Classification** (predict categorical label). **Decision trees** are a widely used **classification** algorithm.
- **Analogy**: Put items into **predefined buckets** (e.g. apple → bin 1, orange → bin 2). The tree implements a hierarchy of **questions** that route each sample to a **leaf** (class label).

Applications: Spam vs non-spam, loan approve/reject, medical diagnosis, etc.

Human-Like Decision Making

Decision trees mirror how **humans** make decisions by a **sequence of criteria**:

Example – Laptop purchase: Criteria in order: Processor (e.g. i7?) → RAM (e.g. 64 GB?) → Disk (e.g. 1 TB?) → GPU. Each “yes/no” or “which value?” narrows the set; finally one laptop (or one class) is chosen.

Example – Loan application: Salaried? → Yes/No. If yes: ITR > 10 lakh? → Approve/Reject. If no: ITR > 20 lakh? → Approve/Reject. Again a **tree of questions** leading to **approve** or **reject**.

Structure of a Decision Tree

Component	Role
Root node	Top of the tree; first decision (one attribute + condition).
Internal (decision) nodes	Non-leaf nodes; each holds an attribute + condition and branches.
Branches / edges	Connect nodes; each branch corresponds to an attribute value (or range).
Leaf nodes	Terminal nodes; contain the class label (e.g. Approve, Reject, Yes, No).

So: **non-leaf** = decision points (attribute + condition); **leaf** = predicted class.

Why Decision Trees Are Popular

1. **White-box / interpretable:** We can **trace the path** from root to leaf and **explain** exactly why a sample got a particular class (e.g. "Salary=Yes, ITR>10L → Approved"). Transparent for users and stakeholders.
 2. **Easy to set up:** Require **minimal data pre-processing** compared to many other algorithms.
-

Two Phases: Induction and Deduction

Phase	Meaning
Induction	Building the tree from training data (how to choose attributes, how to split). This is the training phase.
Deduction	Using the built tree to predict for new/unlabeled data. Start at root, follow branches by attribute values, end at a leaf → that leaf's label is the prediction.

Building a Tree (Induction) – High-Level

- **Input:** Tabular training data (e.g. Refund, Marital status, Taxable income; class = Cheat Yes/No).
- **Idea:** At each node, **choose an attribute** and **split** the data so that **child nodes** become as **pure** (homogeneous) as possible — ideally all one class.
- **Process:** Start at root → pick "best" attribute → split into children → repeat for impure children until a **stopping condition** (e.g. pure node, no more attributes, or pre-pruning). Details (how to choose "best" attribute, how to split) come in later videos.

Example (simplified): Root = Refund. Branch "Yes" → all "No" cheat → **leaf: No**. Branch "No" → mixed → split again by Marital status (e.g. Single/Divorced vs Married) and possibly Taxable income (e.g. >80K) to get leaves Approve/Reject or Yes/No.

Using the Tree (Deduction)

- **Input:** One **test** record (only **X**; no class label).
- **Process:** Start at **root**; check the attribute value (e.g. Refund = No) → follow the matching branch → next node; repeat until you reach a **leaf**.
- **Output:** The **class label** at that leaf is the **prediction**.

Example: Refund=No → Marital status=Married → (e.g. no further split) → **Leaf: No** (e.g. "not cheat" or "reject"). Prediction = No.

Summary

- **Decision tree** = tree of **decision nodes** (attribute + condition) and **leaf nodes** (class labels).
Branches = attribute values.
- **Induction** = building the tree from data; **deduction** = predicting by traversing from root to leaf.
- **White-box** and **easy to set up**; good for interpretation and quick prototyping.

Use this note for structure of a decision tree, induction vs deduction, and tracing a simple prediction path.

Finding the Best Split – Machine Learning (Module 4)

Learning Objectives

By the end of this video you will:

1. **Define node impurity** and **pure vs impure** (homogeneous vs heterogeneous) nodes.
 2. **State** the goal: choose the attribute that leads to the **most homogeneous (pure) child nodes**.
 3. **Define** and **compute entropy** for a node; relate to **information gain**.
 4. **Define** and **compute Gini index** (and **Gini split**).
 5. **Define misclassification error** and compare all three (entropy, Gini, misclassification) for binary classification.
-

Three Questions in Tree Construction

1. **Which attribute** to choose for splitting (e.g. at root)?
2. **How** to split that attribute (multi-way, binary, by threshold)? (Depends on attribute type — see “Art of the Split.”)
3. **When** to stop (stopping condition)? (See “From Induction to Application.”)

This note focuses on **question 1: which attribute to choose**.

Goal: Homogeneous (Pure) Child Nodes

- After a split we get **child nodes**. We want them to be as **pure** as possible:
 - **Pure/homogeneous:** (Almost) all tuples in the node are of **one class** (e.g. all C0 or all C1). Then we can **stop** and assign that class at a leaf.
 - **Impure/heterogeneous:** Mix of classes (e.g. 5 C0 and 5 C1). We **cannot** decide; we need to **split further**.

So we **choose the attribute** that **most reduces impurity** (or equivalently, most increases “purity”) in the children.

Measuring Impurity

- **Impurity measure:** A function that takes the **class distribution** in a node (e.g. proportions p_1 , p_2 for two classes) and returns a **number**.

- **Pure node** → impurity = **0** (or minimum).
- **Impure node** (e.g. 50–50) → impurity = **maximum** (for that measure).

We **compare** attributes by the **impurity of the resulting child nodes** (often using a **weighted average** by number of samples in each child). **Best attribute** = one that gives the **lowest** (weighted) impurity in children, or **largest reduction** from parent.

1. Entropy

- From **information theory**: measures **uncertainty / randomness / chaos**.
- **High entropy** → high impurity (mixed classes). **Low entropy** → low impurity (one class dominates).
- **Formula (binary or multi-class):** $\text{Entropy} = - \sum p_i \log_2(p_i)$ where p_i = proportion of class i in the node.

Example: Node with 3 of class A and 7 of class B → $p_A = 0.3$, $p_B = 0.7$ → Entropy ≈ 0.88 .

Pure node: One class has proportion 1, others 0 → Entropy = 0. **Maximum entropy (binary):** $p = 0.5$ → Entropy = 1.

Information Gain (used with Entropy)

- **Information Gain (IG)** of an attribute = **Entropy(parent) – weighted average of Entropy(children)**.
 - **Larger IG = better** attribute (more reduction in uncertainty). We **choose the attribute with highest IG**.
-

2. Gini Index

- **Gini** = probability that a **randomly chosen** element in the node is **misclassified** if we label it according to the class distribution in the node.
- **Formula:** $\text{Gini}(\text{node}) = 1 - \sum p_j^2$ (sum over classes; p_j = proportion of class j).

Pure node: One $p_j = 1$ → Gini = 0. **Binary, 50–50:** Gini = 0.5.

Gini Split

- Like IG for entropy: we compute **weighted average of Gini** of child nodes. **Higher** Gini (or we use **reduction** in Gini from parent) → we can rank attributes. In many implementations we **minimize** weighted Gini of children (i.e. choose split that **reduces** Gini the most). (Some software uses "Gini gain" or "Gini impurity decrease.")

Used in: CART; often **default** in scikit-learn for decision trees.

3. Misclassification Error

- **Error(node) = $1 - \max(p_i)$** i.e. 1 minus the proportion of the **majority** class.
- **Pure node:** $\max(p_i) = 1 \rightarrow \text{Error} = 0$. **Binary, 50–50:** Error = 0.5.

Comparison (binary classification, horizontal axis = proportion of one class):

- **Entropy:** 0 at $p=0$ and $p=1$; **maximum 1** at $p=0.5$.
- **Gini:** 0 at $p=0$ and $p=1$; **maximum 0.5** at $p=0.5$.
- **Misclassification error:** 0 at $p=0$ and $p=1$; **maximum 0.5** at $p=0.5$.

So for **binary** classification: **Entropy** range $[0, 1]$; **Gini** and **misclassification error** range $[0, 0.5]$.

How We Use These

- At each node we have a set of **candidate attributes** (e.g. A, B, C).
- For each attribute we **split** the data (according to that attribute's type and chosen split). We get **child nodes**.
- We compute **impurity** (entropy, Gini, or misclassification) for each child and then a **combined** measure:
 - **Entropy** $\rightarrow$ **Information Gain** = Entropy(parent) – weighted_avg(Entropy(children)). **Choose attribute with maximum IG.**
 - **Gini** $\rightarrow$ **Gini split** (weighted Gini of children) or **decrease in Gini**; choose attribute that **minimizes** child Gini (or maximizes decrease).
 - **Misclassification** $\rightarrow$ similarly compare weighted error of children; choose attribute that **minimizes** it.

We **iterate** until a **stopping condition** (pure node, no gain, or pre-pruning).

Summary

- **Pure node** = one class (impurity 0); **impure** = mixed (impurity > 0). We want **splits that create purer children**.
- **Entropy:** $-\sum p_i \log_2(p_i)$. Use **Information Gain** = Entropy(parent) – weighted Entropy(children). **Max IG** $\rightarrow$ best attribute. Used in ID3, C4.5.
- **Gini:** $1 - \sum p_i^2$. Use **Gini split** (weighted Gini of children). **Minimize** child Gini (or maximize decrease). Used in CART; default in many libraries.
- **Misclassification error:** $1 - \max(p_i)$. Simpler; ranges 0–0.5 for binary.

Use this note for computing entropy/Gini/error and for choosing the best attribute by IG or Gini.

The Art of the Split – Handling Different Attributes – Machine Learning (Module 4)

Learning Objectives

By the end of this video you will:

1. **State** that decision tree construction follows a **greedy** strategy (local optimization, no backtracking).
 2. **List** the three questions: which attribute, how to split, when to stop.
 3. **Describe** how to split **nominal** attributes: **multi-way** vs **binary** split.
 4. **Describe** how to split **ordinal** attributes (order must be preserved when grouping).
 5. **Describe** how to split **continuous** attributes: **discretization** (bins) vs **binary** split (e.g. threshold).
-

Greedy Strategy

- At **each** node we choose the **best attribute (and split) at that moment**, using a **local** criterion (e.g. maximum information gain, or minimum Gini in children).
 - We **do not** look ahead or **backtrack**. Once we split on attribute A at the root, we do not later replace A with B at the root.
 - **Consequence:** Decision trees **do not** guarantee a **globally optimal** tree; they optimize **locally** at each step.
-

Three Questions (Recap)

1. **Which attribute** to choose? → Use impurity reduction (e.g. information gain, Gini) — see “Finding the Best Split.”
 2. **How to split** that attribute? → **Depends on attribute type** (this note).
 3. **When to stop?** → Stopping conditions — see “From Induction to Application.”
-

Splitting by Attribute Type

We treat attributes as **nominal**, **ordinal**, or **continuous** (interval/ratio grouped as continuous for splitting).

1. Nominal Attribute

- **Property:** Only **distinctness** (same/different). No order, no magnitude.
- **Options:**

Multi-way split

- **One branch per distinct value** (e.g. Car type: Family, Sports, Luxury → 3 branches). All samples with that value go to that child.
- **Pros:** Simple, intuitive. **Cons:** Many branches → few samples per child; can hurt performance.

Binary split

- Partition the **set of values** into **two non-overlapping subsets**; two branches. E.g. {Family, Sports} vs {Luxury}, or {Family} vs {Sports, Luxury}.
- **Challenge:** For k values there are $2^k - 1$ ways to form two subsets. Finding the **optimal** binary split can be expensive; heuristics are used.

2. Ordinal Attribute

- **Property:** Values can be **ordered** (e.g. small < medium < large). **Order** must be respected when grouping.
- **Options:** Again **multi-way** (one branch per value) or **binary**.
- **Rule for binary (or grouping):** When combining values into one branch, **do not break the order**. **Valid:** {Small, Medium} vs {Large}. **Invalid:** {Small, Large} vs {Medium} (order broken).

3. Continuous Attribute

- **Property:** Infinite possible values (e.g. age, taxable income, temperature).
- **Options:**

Discretization (multi-way)

- Define **bins** (e.g. Age: 1–25, 26–50, 51–75, 76–100). Each bin → one branch; samples go to the bin that contains their value.
- **Binary** version: e.g. "Age ≤ 25" vs "Age > 25" (one threshold).

Binary split (threshold)

- Choose a **threshold** (e.g. Taxable income > 80K). Two branches: **yes** and **no**. Very common in practice (e.g. CART).
- We can also use **multiple thresholds** (several bins) like discretization.

Summary Table

Attribute type	Multi-way split	Binary split	Constraint
Nominal	One branch per value	Partition values into 2 subsets	Subsets disjoint
Ordinal	One branch per value	Partition into 2 subsets	Preserve order when grouping
Continuous	Bins (discretization)	Single (or multiple) threshold(s)	—

Exam-Oriented Points

- Tree construction is **greedy** (local choice, no backtracking).
- **Nominal**: Multi-way or binary; optimal binary split is combinatorial ($2^{(k-1)}-1$).
- **Ordinal**: Multi-way or binary; **order must be preserved** in groupings.
- **Continuous**: Discretization (bins) or binary (e.g. "attribute > threshold").

Use this note for "how do we split nominal/ordinal/continuous" and "what is greedy strategy."

From Induction to Application – Pruning, Overfitting and Advantages – Machine Learning (Module 4)

Learning Objectives

By the end of this video you will:

1. List the **stopping conditions** for decision tree construction (pure node, no gain, early termination).
 2. **Describe pre-pruning** (early termination) and typical **thresholds** (min samples, max depth, min improvement).
 3. **Relate overfitting** and **underfitting** to tree **complexity** (e.g. depth) and to **train vs test error**.
 4. **State** that we want a **generalized** tree (sweet spot) and **avoid** overfitting and underfitting.
 5. List main **advantages** of decision trees (cost, speed, interpretability, comparable accuracy).
-

Recap: Tree Induction

- **Three questions:** (1) Which attribute to choose? (2) How to split it? (3) When to stop?
 - **Process (e.g. ID3-style):** Top-down. At each node: choose **best attribute** (e.g. by information gain), **split** into children, **assign** training examples to leaves. **Recurse** on each child until a **stopping condition** is met. **Greedy:** no backtracking.
-

Stopping Conditions

We stop expanding a node when **any** of the following holds:

1. Pure node

- All tuples in the node belong to **one class** → impurity = 0. We can **stop** and make that node a **leaf** with that class label.

2. No significant gain

- **No attribute** gives a **significant** reduction in impurity (e.g. information gain or Gini decrease below a threshold). Further split is not justified → **stop** and declare a leaf (e.g. majority class).
- **Records indistinguishable:** Algorithm cannot separate them with available attributes → stop.

3. Early termination (pre-pruning)

- We **manually** stop growth using **thresholds**, e.g.:
 - **Minimum samples for split**: Do not split if the node has too few samples (avoids fitting noise).
 - **Maximum tree depth**: Do not grow beyond **n** levels (limits complexity and prediction cost).
 - **Minimum improvement**: Do not split unless impurity **decreases by at least** a given amount.

These help **avoid overfitting** and keep the tree **generalizable**.

Overfitting and Underfitting in Decision Trees

- **Model complexity** for a tree can be thought of as **depth** (or number of nodes). Deeper tree = more complex.
- **Train error** tends to **decrease** as we allow more depth. **Test error** first **decreases** then **increases** (we overfit).

Underfitting

- **Too shallow / too simple** tree (e.g. depth 1). **High train error, high test error**. Model cannot capture the pattern in the data.

Overfitting

- **Too deep** tree; it **memorizes** training data (including noise and outliers). **Low train error, high test error**. Fails on new data.
- **Analogy**: Student memorizes exact practice answers; real exam has different questions → poor performance.

Sweet spot

- **Moderate** depth (and/or other constraints) where **train and test error** are **both low and close**. Tree **generalizes** well.

Ways to reach the sweet spot: **Pre-pruning** (max depth, min samples, min improvement), and later **post-pruning** (not detailed here). Goal = **generalization**.

Advantages of Decision Trees

Advantage	Meaning
Relatively inexpensive to construct	Time/complexity of building the tree is often lower than many other algorithms.
Fast prediction	For a new sample we only traverse from root to one leaf (few comparisons). Very fast at inference.
Easy to interpret	White-box : we can trace the path and explain why a prediction was made (root → ... → leaf).
Comparable accuracy	In many problems, decision trees achieve accuracy comparable to other classifiers.
Minimal pre-processing	Often need less data cleaning and scaling than distance-based or gradient-based methods.

Summary

- **Stopping conditions**: (1) Pure node, (2) No significant impurity reduction / indistinguishable records, (3) Pre-pruning (min samples, max depth, min improvement).
- **Overfitting** = too complex tree, low train error and high test error; **underfitting** = too simple, high train and test error. Aim for **generalization** (sweet spot).
- **Pre-pruning** (early stopping) helps avoid overfitting.
- **Advantages**: Inexpensive to build, fast prediction, interpretable, often comparable accuracy, minimal pre-processing.

Use this note for "when do we stop building the tree," "overfitting/underfitting in trees," and "advantages of decision trees."